Tree

A few points:

So far, we have discussed general tree, binary tree and binary search tree. We have also discussed at length two different implementations of the tree – node and dynamic memory, node and array based.

In this discussion, we will focus on two aspects.

a) Make better use of pointers in node and dynamic memory-based tree structure.
b) Evaluate an expression by creating an expression tree based on an expression in postfix notation.

Threaded binary tree:

In a tree of n nodes, there are only n-1 edges or branches. In a node based implementation, each node has 2 pointers. We require only n-1 pointers to point to nodes – these are called structural pointers. So, as many as n + 1 pointers will be grounded.

Based on a particular selected order of traversal (say inorder), can we, instead of grounding the left pointer, make it point to the predecessor? Similarly, instead of grounding the right pointer, can we make it point to the successor? Such a pointer is called a thresad.

How do we make out whether the pointer is a structural pointer or a thread?

To facilitate this, we may require two bit fields.

Check the structure of the node in the header file : tree.h in the folder threaded.

Lthread is 1 if left is thread and a structural pointer. Similarly, rthread is 1 if right is a thread and not a structural pointer. Otherwise, they will be 0.

Interface:

```
struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
        int lthread_;
        int rthread_;
};
typedef struct node node_t;
```

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;
```

The interface of the tree has not changed.

```
void init(tree_t *ptr_tree);

void deinit(tree_t *ptr_tree);

void disp(tree_t *ptr_tree);

void insert(tree_t *ptr_tree, int key);
```

Implementation:

Initialize :

The root is grounded.

```
void init(tree_t *ptr_tree)
{
        ptr_tree->root_ = NULL;
}
```

Insert:

There is a helper function which creates a node.

Note that a new node is created. If the allocation fails, this function gives an error message and exits the program.
To the new node, key is copied.
Always the node is added as a leaf. The pointers are grounded.
We assume that the pointers are threads. We could also think otherwise and suitably take care in the insert routine.

```
static node_t* create_node(int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        if(temp == NULL)
        {
                printf("no more memory\n"); exit(1);
        }
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        temp->lthread_ = temp->rthread_ = 1;
        return temp;
}
```

This insert function calls the function insert_key to create a node and insert the node and adjust the structural and thread pointers.

```
void insert(tree_t *ptr_tree, int key)
{
        ptr_tree->root_ = insert_key(ptr_tree->root_, key);
}
```

This is quite a long function.
We will discuss the different parts at length.

```
static node_t* insert_key(node_t* root, int key)
{
        node_t *temp = create_node(key);
        node_t *parent = NULL;
        node_t *curr = root;
         while (curr != NULL)
         {
               parent = curr;
               if (key < curr->key_)
               {
                  if (curr->lthread_ == 0)
                              curr = curr->left_;
                  else
                              break;
               }
              else
              {
                   if (curr->rthread_ == 0)
                              curr = curr->right_;
                   else
                              break;
```

```
            }
        }
    if (parent == NULL)
    {
       return temp;
    }

    /* Insert as left child */
    if (key < parent->key_)
    {
       temp->left_ = parent->left_;
       temp->right_ = parent;

       parent->left_ = temp;
       parent->lthread_ = 0;
    }

    /* Insert as right child */
    else
    {
       temp->right_ = parent->right_;
       temp->left_ = parent;

       parent->right_ = temp;
       parent->rthread_ = 0;
    }

    return root;
}
```

- Create a node and initialize local variables
  Observe that the node does not have a parent pointer. It is a local variable.
  ```
  node_t *temp = create_node(key);
  node_t *parent = NULL;
  node_t *curr = root;
  ```

- Take care of the base case when the first node is added and the tree is empty.
  ```
  if (parent == NULL)
  {
     return temp;
  }
  ```

- This is the loop to find the position to insert in the binary search tree. When the subtree is not empty, move along the tree. Remember that we always attach a new node as a leaf. Update the

parent before changing current. Based on the key comparison, decide to move left or right. If we have reached a thread, it means that we have reached a leaf.
Removing the break in the code is left as an exercise to you.

```
while (curr != NULL)
{
      parent = curr;
      if (key < curr->key_)
      {
         if (curr->lthread_ == 0)
               curr = curr->left_;
         else
                break;
      }
      else
       {
               if (curr->rthread_ == 0)
                        curr = curr->right_;
               else
                        break;
       }
  }
```

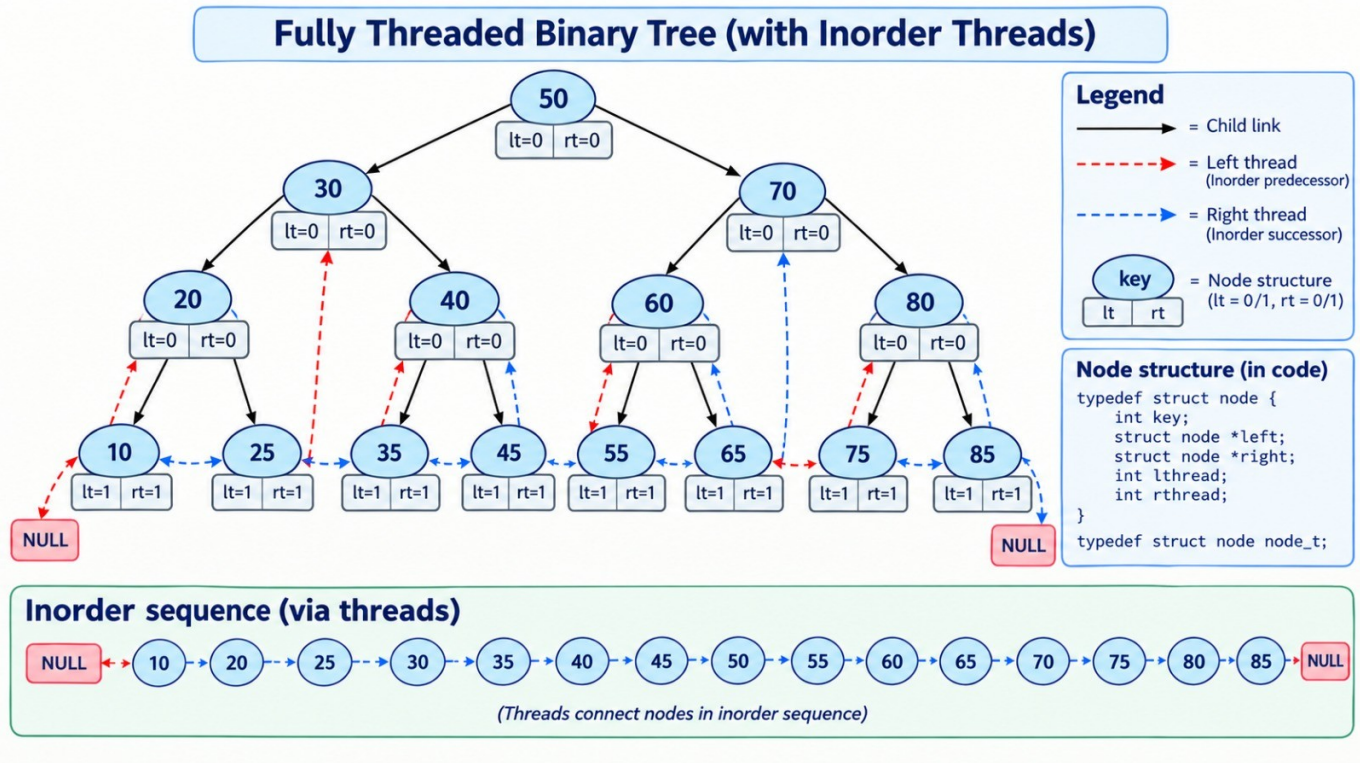
- Once the loop is exited, we have found a position to insert. We should find whether the insertion is happening in the left subtree or the right subtree with respect to the parent.
  To decide that, key is compared with the key of the parent.

```
  if (key < parent->key_)
  {
     temp->left_ = parent->left_; // why temp->left_ not NULL?
     temp->right_ = parent; // parent is the successor; right_ is a thread

     parent->left_ = temp; // attach to the left subtree; so this is a structural pointer
     parent->lthread_ = 0; // and hence not a thread
  }
  else
  {
     temp->right_ = parent->right_; // why temp->right_ not NULL?

     temp->left_ = parent; // parent is the successor; left_ is a thread


     parent->right_ = temp; // attach to the right subtree; so this is a structural pointer
     parent->rthread_ = 0; // and hence not a thread
  }
  return root;
```



The left of the leftmost node and the right of the rightmost node are threads and may point to the root node or may be grounded.

Display the tree in inorder:

This function disp calls the inorder function. Note that the inorder function is not recursive.

```
void disp(tree_t *ptr_tree)
{
        printf("Tree : ");
        inorder(ptr_tree->root_);
        printf("\n");
}
```

There is a helper function to find the leftmost node in the tree.
If the tree is empty, there is no leftmost node.
Otherwise keep traversing left. Remember no pointers are grounded. So, we will come to know that the left subtree is empty if the left is a thread.

```
static node_t *leftmost(node_t *temp)
{
   if (temp == NULL)
      return NULL;

   while (temp->lthread_ == 0)
      temp = temp->left_;

   return temp;
}
```

This function inorder finds the leftmost node without a left subtree.
It will display the key and then follow the right if is is a thread.
Otherwise it is a structural pointer. Take one step right and then again find the leftmost node.

```
static void inorder(node_t *root)
{
   node_t *temp = leftmost(root);

   while (temp != NULL)
   {
      printf("%d ", temp->key_);

      if (temp->rthread_ == 1)
         temp = temp->right_;
      else
         temp = leftmost(temp->right_);
   }
}
```

This completes the discussion of threaded binary tree.